Transcrição

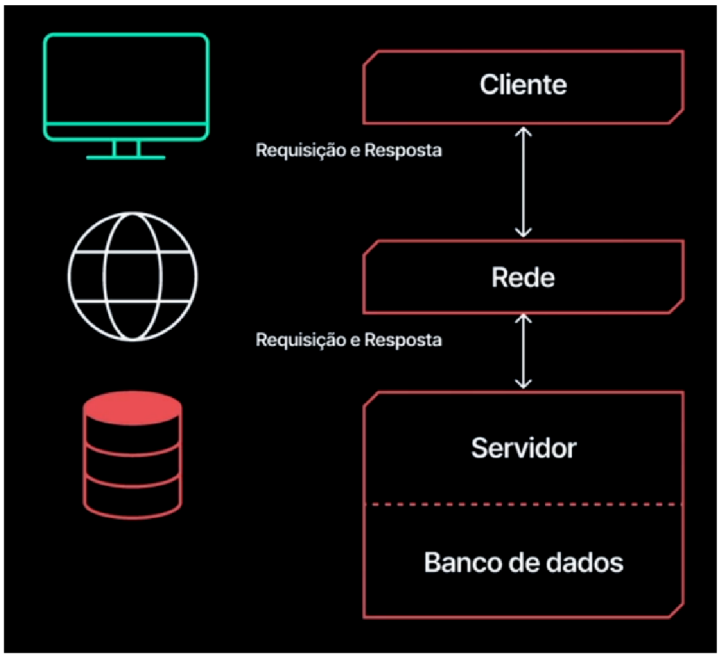
Após modelarmos todo o sistema que desejamos construir, o próximo passo é determinar como ele será **desenvolvido**. Imaginemos que estamos em uma *startup* que tem o objetivo de **criar uma plataforma de cursos e aulas online**, voltada para

Para testar nosso modelo de negócio, é essencial desenvolver rapidamente a plataforma e disponibilizá-la para alguns testadores beta, a fim de obter *feedback* e analisar os resultados. Nesse cenário, qual seria a arquitetura mais adequada para ser utilizada?

Abordagem monolítica

Como a agilidade no desenvolvimento é um fator crucial neste processo, optamos por adotar uma **abordagem simplificada**, conhecida como **monolítica**. Nessa

abordagem, ao analisarmos o fluxograma de funcionamento de uma aplicação como essa:

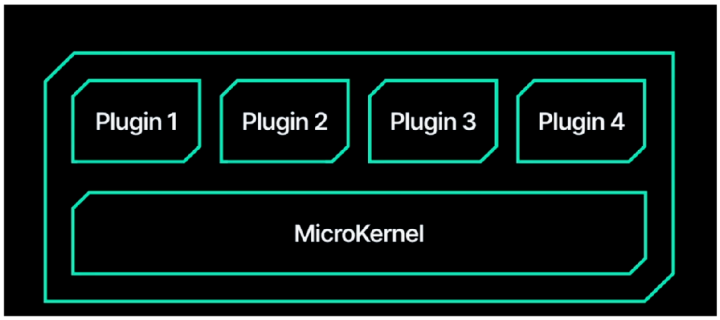


Temos um dispositivo cliente, no qual a pessoa usuária se conecta à rede para acessar a plataforma. A plataforma, por sua vez, está hospedada em um servidor, que se comunica com um banco de dados. Esse modelo segue a estrutura tradicional de **requisição** e **resposta**: a pessoa usuária realiza uma interação, e a página correspondente é renderizada no navegador ou no aplicativo móvel.

Em uma arquitetura monolítica, do lado do servidor e banco de dados, **construímos toda a aplicação** como se fosse um único bloco de código. Ou seja, *back-end*, *front-end* e banco de dados estão combinados em um único bloco de código. A vantagem é que conseguimos **desenvolver rapidamente**

esse tipo de aplicação usando essa arquitetura. No entanto, quando nossa plataforma começa a crescer e passamos dos *beta testers* para o mercado, podemos enfrentar alguns **gargalos**.

Essa arquitetura é muito utilizada na **construção de sistemas operacionais**.



Temos um microKernel de um sistema operacional e vários *plugins* associados dentro de uma arquitetura monolítica.

No entanto, voltando ao caso da nossa plataforma, quando ela evolui e sai do estágio de teste, aumentando o número de pessoas usuárias, é importante que tenhamos um **código fácil de manter**. Se percebemos um erro ou uma funcionalidade que não está funcionando conforme esperado, ou se o desempenho não está satisfatório, como proceder?

Na arquitetura monolítica, se precisarmos mudar algo, teremos que **alterar toda a base de código** e **reimplantar toda a solução em produção**, o que pode ser custoso e demorado. Para facilitar a construção de

monolíticos, podemos adotar uma abordagem de **dividir a aplicação em camadas**.

Uma plataforma pode ser organizada em diferentes camadas, começando pela ***interface* da pessoa usuária**, que será responsável por todas as funcionalidades de interação com a pessoa usuária. Além disso, pode haver uma camada dedicada à implementação da **lógica de negócios**, englobando todas as funcionalidades previstas nos requisitos da plataforma. Também é possível contar com uma camada voltada para o **gerenciamento do banco de dados**, garantindo o acesso e a recuperação rápida das informações armazenadas.

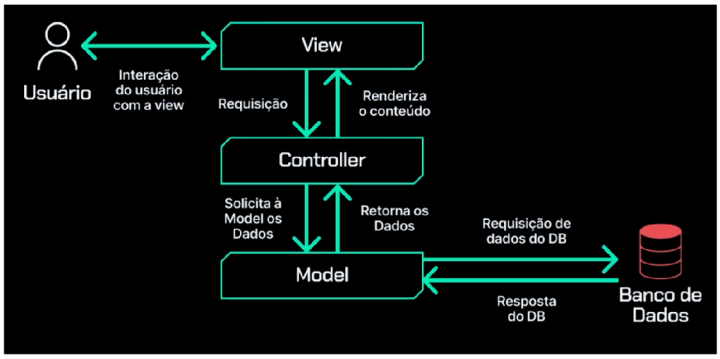
- Interface de usuário
- Lógica de negócios
- Sistema de gerenciamento de banco de dados

Essa abordagem melhora o desempenho, pois conseguimos **refatorar** e melhorar o código com base nessas camadas.

Apresentando a arquitetura MVC

Dentro dessa abordagem de desenvolver uma arquitetura em camadas, há um modelo famoso e amplamente utilizado no

desenvolvimento, que é a **arquitetura MVC** ("*Model-View-Controller*").



Definindo as camadas do MVC

Nessa arquitetura, temos uma camada chamada "*View*" ("Visão ou Interface"), que fornece toda a **visão da plataforma para a pessoa usuária**. A pessoa usuária interage com a camada View.

Por outro lado, temos a camada "*Controller*" ("Controlador"), que **recebe as requisições** da pessoa usuária e **renderiza o conteúdo**, carregando as páginas e percebendo as demandas da pessoa usuária. Além disso, temos a camada "Model" ("Modelo"), que **solicita dados**, por exemplo, ao banco de dados. Se a pessoa usuária deseja carregar um vídeo de matemática, a camada Model solicita o vídeo sobre função do primeiro grau para carregá-lo na página da pessoa usuária.

Essa é a arquitetura MVC, que proporciona uma **divisão clara de tarefas** entre as

camadas. Com isso, temos um código mais bem **estruturado**, facilitando a reparação de funcionalidades rapidamente, o que melhora a organização e o crescimento do código. Com o tempo, além do aumento do número de pessoas usuárias, podemos ter demandas para incluir novas funcionalidades. A tecnologia muda, e as expectativas das pessoas em relação a uma aplicação também mudam. Isso facilita o crescimento da base de código.

Avaliando as abordagens arquiteturais

Existem outras abordagens arquiteturais que exploraremos mais adiante. Antes de prosseguir, é importante ressaltar: **qual é a melhor arquitetura para minha aplicação?** Essa pergunta não tem uma resposta definitiva, pois depende do que estamos construindo, de como vemos o futuro e a vida útil da aplicação, seu tempo e evolução.

Muitas vezes, uma aplicação não adota uma única arquitetura, mas sim uma **arquitetura híbrida**, com partes do código implementadas em MVC e outras usando diferentes abordagens arquiteturais. É essencial avaliar bem o que esperamos em termos de comportamento, componentes estruturais e a interação entre eles para

decidir a abordagem a ser utilizada na construção da arquitetura.

Próximos passos

Vamos continuar nossa jornada pela exploração de abordagens arquiteturais.

Transcrição

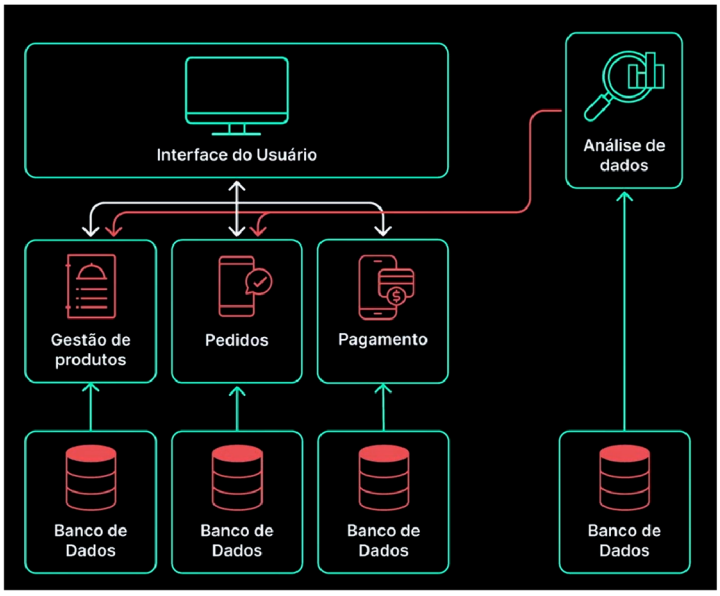
Analisamos algumas abordagens arquiteturais, e agora consideramos um outro cenário: um ***e-commerce***. De forma geral, um e-commerce lida com uma alta demanda de pessoas usuárias, com picos de acesso em datas promocionais, como a *Black Friday* e outras datas festivas.

Se continuarmos utilizando uma arquitetura baseada no modelo de camadas, como o modelo MVC, o desempenho da aplicação e a experiência das pessoas usuárias podem **não ser satisfatórios** à medida que a base de clientes cresce de forma significativa. Por isso, é necessário adotar uma abordagem arquitetural diferente.

Arquitetura de microsserviços

Uma das abordagens que se destaca para atender esse tipo de demanda é a arquitetura baseada em **microsserviços**. Nessa arquitetura, dividimos a aplicação em

serviços independentes que interagem entre si utilizando APIs.



Por exemplo, em um e-commerce simplificado, temos alguns microsserviços: gestão de produtos, pedidos, pagamento e análise de dados. O serviço de *interface* é importante, pois é onde a pessoa usuária consegue comprar itens, colocá-los no carrinho e fazer pedidos.

Quando a pessoa usuária interage com o serviço de interface, este, a partir de alguma demanda da pessoa usuária, busca na base de dados, usando o serviço de gestão de produtos, se há algum item disponível para que o pedido possa ser feito. Se o item estiver disponível, o pedido é lançado no banco de dados diretamente pelo microsserviço de pedidos.

Além disso, temos um serviço de pagamentos e, se formos as pessoas

administradoras do e-commerce, também temos um serviço de análise de dados para verificar vendas, itens mais vendidos, perfis de clientes, entre outros.

Vantagens da arquitetura de microsserviços

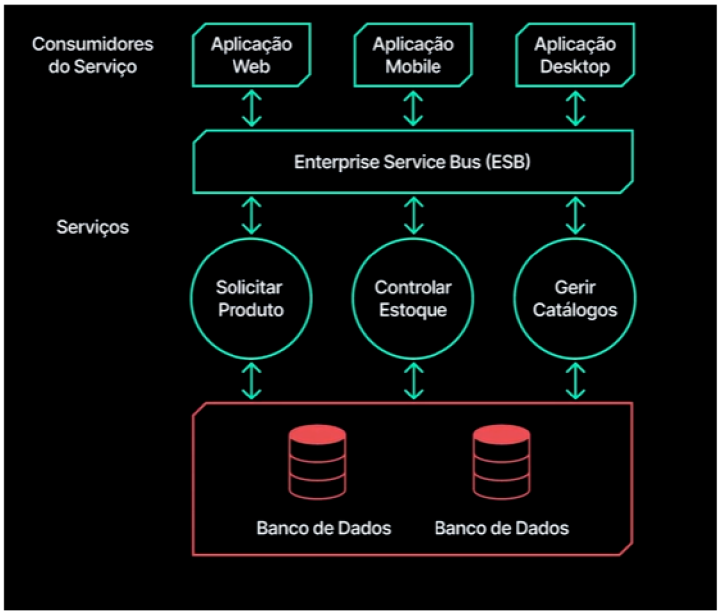
A principal vantagem dessa abordagem arquitetural é a **divisão clara** dos serviços. Se precisarmos atualizar o módulo de pagamento, conseguimos fazer isso sem afetar os demais serviços. Da mesma forma, podemos fazer ajustes na gestão de produtos ou incluir novas funcionalidades como novos serviços, com grande flexibilidade para acomodar essas mudanças. Isso facilita a manutenção do código e a evolução da aplicação.

Outra vantagem é que os microsserviços podem ser **escritos em diferentes linguagens de programação**. Por exemplo, o microsserviço de pagamentos pode ser escrito em Java, enquanto o de análise de dados pode ser em Python. Isso não causa

funcionam como **componentes independentes**, ao contrário de uma arquitetura monolítica, que exige o uso da mesma linguagem para a maior parte do código.

Abordagem baseada em eventos

Além da abordagem de microsserviços, podemos considerar uma abordagem baseada em **eventos**.



Nessa abordagem, temos os consumidores do serviço, que são os clientes, e cada interação com a aplicação, como comprar um produto ou verificar sua disponibilidade, gera um evento. Esse evento é respondido por diferentes partes da aplicação de forma específica.

A vantagem dessa arquitetura é que os eventos podem ser processados de maneira **assíncrona**, proporcionando um alto nível de **desacoplamento** entre os componentes. Isso oferece flexibilidade para inserir novas funcionalidades, alivia a carga de processamento e melhora o desempenho,

além de aumentar a resiliência da plataforma.

A **resiliência** significa que, se houver uma falha em um serviço, como a solicitação de um produto, isso não afetará o funcionamento da gestão dos catálogos. Ao contrário de uma arquitetura em camadas, onde um mau comportamento em uma camada pode afetar as camadas superiores, aqui temos uma compartimentalização do funcionamento da aplicação.

Arquitetura orientada a serviços

O acrônimo SOA, que significa *"Service Oriented Architecture"* ("Arquitetura Orientada a Serviços"). Essa arquitetura é baseada em serviços e pode incluir arquiteturas baseadas em eventos e microsserviços.

> Um **serviço** é uma função independente bem definida que oferece uma **funcionalidade específica**, como a solicitação de um produto, controle de estoque ou gestão de catálogo.

Uma das características interessantes da arquitetura baseada em eventos é a **flexibilidade** e **resiliência**, mesmo com o

crescimento da aplicação. No entanto, ainda dependem de uma infraestrutura física comum em muitas arquiteturas.

Próximos passos

Para reduzir essa dependência, podemos considerar abordagens ***serverless*** ("sem servidor"). Vamos analisar essas abordagens na sequência?

06

Para saber mais: padrões de projeto na arquitetura de microsserviços

Vimos que **Microsserviços** é uma arquitetura de software onde uma aplicação é dividida em serviços pequenos e independentes, cada um responsável por uma função ou tarefa específica.

Cada microsserviço é autossuficiente e pode ser desenvolvido, implantado e escalado de maneira separada. Esses serviços se comunicam entre si por meio de APIs, o que permite maior flexibilidade e agilidade no desenvolvimento, além de facilitar atualizações e manutenção sem impactar o sistema como um todo.

Caso deseje se aprofundar sobre o uso de microsserviços como arquitetura na construção de uma aplicação, o curso "Microsserviços: Padrões de Projeto" (https://cursos.alura.com.br/course/microsservicos-padroes-projeto) é uma excelente opção. Nele, você aprenderá como aplicar padrões de projeto para resolver desafios comuns no desenvolvimento de microsserviços, como comunicação entre serviços, gerenciamento de dados e orquestração. Com uma abordagem prática e focada em soluções reais, o curso

oferece uma visão completa de como construir e manter uma arquitetura de microsserviços.

Além deste curso, temos aqui na plataforma diversas opções focadas em linguagens como Java, .NET e outras tecnologias que complementam a formação em microsserviços. Com isso, você pode ter uma experiência completa de aprendizado, aprofundando-se tanto nas práticas de arquitetura quanto nas ferramentas e linguagens mais utilizadas para implementá-las.

Transcrição

Uma das questões que levantamos anteriormente foi a necessidade de uso de uma infraestrutura dedicada para nossa aplicação, considerando a utilização de algum servidor ou instância na nuvem, mesmo com arquitetura de microsserviços,

chamada ***serverless***.

Entendendo a abordagem *serverless*

> É uma abordagem em que **não** utilizamos uma infraestrutura dedicada ao funcionamento da nossa aplicação.

Podemos nos questionar como a aplicação funcionará dessa forma. Basicamente, são serviços disponibilizados por provedores de computação em nuvem, como a AWS, Azure e Google Cloud, entre outros, nos quais criamos uma **função** que responderá a

eventos. É uma forma de arquitetura baseada em eventos.

Toda vez que uma requisição chega a esse serviço dentro dessa função serverless em um provedor de nuvem, a função é executada e a pessoa usuária obtém a resposta desejada.

Quando utilizar essa abordagem?

Utilizamos em situações com **demanda imprevisível**. Por exemplo, podemos ter uma demanda maior em um determinado período e, em outros períodos, uma demanda muito pequena ou quase nula. Nesse modelo serverless, pagamos basicamente **conforme o período de uso**, o que é uma vantagem em relação a ter uma infraestrutura dedicada à aplicação.

AWS

No console da AWS, o serviço serverless é conhecido como **Lambda**. Ao digitarmos "lambda" no menu superior esquerdo, na barra de pesquisa do console, aparece a primeira opção, que é o Lambda, identificado por uma letra grega como símbolo. Existem exemplos de funções

Lambda já implementadas no caso do instrutor.

Ao clicar na primeira opção "data-science-aws-alura", o código da aplicação que responde a um evento é carregado. Ao subir a página, temos uma visão geral dessa função. É necessário usar outros serviços da AWS para receber uma requisição da pessoa usuária e responder a essa requisição, para que a função seja executada conforme um **evento**.

Azure Functions

Não apenas na AWS, mas também na **Azure**, temos o que chamamos de Azure Functions (https://azure.microsoft.com/pt-br/products/functions/), que são funções Azure que executam códigos sem servidores de maneira similar, respondendo a eventos.

Há exemplos de arquitetura na documentação da Azure (https://learn.microsoft.com/pt-br/azure/architecture/web-apps/idea/scalable-ecommerce-web-app), que fornecem uma série de documentações com exemplos de arquitetura de aplicações, onde o uso das Azure Functions é demonstrado. Parte do serviço dessa aplicação será executada de maneira serverless.

É importante destacar que podemos usar o serverless como parte da estrutura da aplicação, não necessariamente totalmente baseada no modo serverless. Essa abordagem serverless se desvencilha do modelo tradicional cliente-servidor.

Próximos passos

Daremos sequência à investigação de diferentes abordagens de estrutura para nossa aplicação, explorando ***pipes and filters*** ("tubos e filtros"). Até mais!

Para saber mais: explorando o mundo serverless com AWS Lambda

No modelo serverless, cada função ou serviço é executado em resposta a eventos específicos, como requisições HTTP, uploads de arquivos ou alterações em bancos de dados. Essa abordagem oferece a capacidade de criar **aplicações altamente escaláveis**, que crescem automaticamente para lidar com picos de demanda e, ao mesmo tempo, reduzem os custos operacionais ao ajustar a utilização dos recursos conforme a demanda diminui. Assim, temos uma infraestrutura mais eficiente e econômica.

O **AWS Lambda** (https://aws.amazon.com/pt/pm/lambda/?), um dos serviços mais populares desse modelo, permite executar código sem se preocupar com a gestão de servidores. Com o Lambda, podemos escrever funções que são acionadas por eventos, e a AWS se encarrega de toda a escalabilidade, gerenciamento de servidores e manutenção da infraestrutura.

Se você quer aprender a desenvolver e implantar funções serverless utilizando AWS Lambda, o curso "AWS Lambda: Código sem se preocupar com a infraestrutura"

(https://www.alura.com.br/curso-online-aws-lambda-codigo-sem-se-preocupar-infra) é a escolha ideal. Ao longo do curso, você explorará todas as etapas de criação de funções, integração com outros serviços e otimização do uso de recursos na nuvem, aproveitando ao máximo o potencial do modelo serverless para criar aplicações escaláveis, eficientes e econômicas.

Transcrição

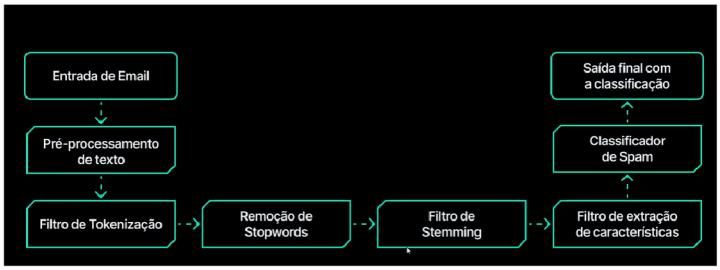
Nessa jornada de análise de uma série de abordagens arquiteturais para uma aplicação, há uma arquitetura mencionada anteriormente chamada *Pipes and Filters* ("Dutos e Filtros"). Que tipo de uso poderíamos fazer com essa abordagem

Aplicações práticas da arquitetura

Em geral, quando trabalhamos com análise e processamento de dados, como no caso do e-commerce, com uma arquitetura de microsserviços, temos um serviço de análise de dados, que é cada vez mais comum nas mais diferentes aplicações.

Temos uma **sequência de etapas bem definidas** pelas quais os dados devem passar para obtermos um resultado final, como a classificação de um cliente ou de um produto e atividades afins.

Entendendo a abordagem *Pipes and Filters*



Para entender bem como funciona essa abordagem de Pipes and Filters, vamos pensar em uma situação do nosso dia a dia: um e-mail que chega na nossa caixa de entrada e passa por uma série de filtros para ser classificado como spam ou como uma mensagem legítima.

Inicialmente, ocorre a entrada de um e-mail. Em seguida, realiza-se o pré-processamento do texto, depois temos a tokenização, seguida pela remoção de *stopwords*, um filtro de stemming, a extração de características e, por fim, a aplicação de um classificador de spam. A partir desse processo, obtemos o resultado final, que indica se o e-mail recebido é considerado spam ou não, determinando, assim, se ele será direcionado para a caixa de spam ou para a caixa de entrada principal.

Observe que temos **diferentes filtros** e, ao mesmo tempo, os dados que saem de um filtro são diretamente encaminhados para o

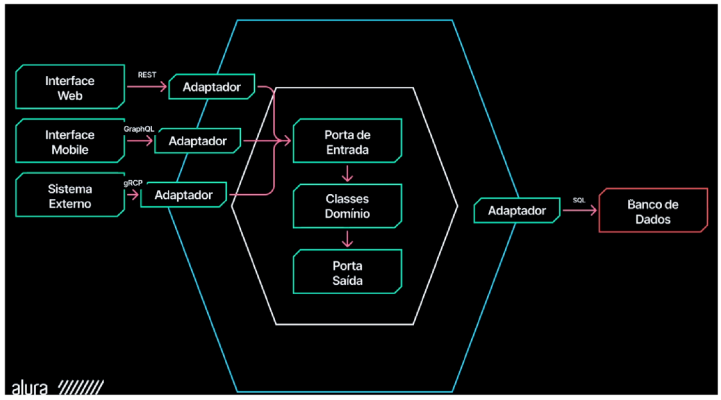
próximo filtro. Esse passo de encaminhamento dos dados é de onde vem o nome *pipes* ("dutos"), como se tivéssemos um encanamento e os dados fossem filtrados por essa série de processos dedicados à atividade final de classificar o e-mail como spam ou não. Poderíamos ter outros tipos de pipes e filtros dentro do processo de análise de dados, de acordo com a atividade e o negócio em que estivermos trabalhando.

Por outro lado, é importante compreender que o mercado de tecnologia é extremamente dinâmico, o que implica não apenas na necessidade de atualizar nossa aplicação, mas também na adaptação aos constantes ajustes dos **serviços** que utilizamos nela. Para garantir o funcionamento adequado, é essencial que a aplicação seja ajustada para manter a **integração** com esses serviços, prevenindo qualquer tipo de falha de funcionalidade ou erro.

Arquitetura hexagonal

No contexto específico de um e-commerce, é necessário integrar um novo método de **pagamento**. Como poderíamos realizar essa integração de maneira simples? Uma abordagem eficaz em termos de arquitetura para facilitar a integração da aplicação com

novas tecnologias e funcionalidades é estabelecer uma separação clara entre a lógica de negócios da aplicação e os sistemas externos, como as tecnologias e serviços de pagamento. Isso pode ser alcançado utilizando uma arquitetura conhecida como **hexagonal**.



A arquitetura hexagonal não é por acaso. Temos, primeiramente, os **adaptadores** que implementam as **portas**, ou seja, as interfaces de comunicação da nossa aplicação com o mundo externo. Dentro, temos o **núcleo**. No núcleo, temos a porta de entrada de dados, classes e domínio, e a porta de saída, que se comunica diretamente com o **banco de dados** e, às vezes, com a própria interface.

Nessa aplicação, a **lógica de negócios está fechada no núcleo**. O mundo externo está conectado à nossa aplicação por meio das portas e dos adaptadores. Com isso, temos um melhor grau de isolamento e conseguimos evoluir melhor, ou seja, evoluir em termos de integração com novas

tecnologias de forma mais fácil. Temos mais flexibilidade para isso.

Após analisarmos diversas arquiteturas, chegamos à conclusão de que cada uma delas atende a finalidades específicas. Ao desenvolver uma aplicação, é preciso lidar com uma série de requisitos e regras de negócio, e a escolha da arquitetura mais adequada é fundamental. O objetivo dessa escolha é garantir a escalabilidade da aplicação, permitindo que ela atenda a uma demanda crescente de pessoas usuárias, além de possibilitar sua evolução contínua.

Considerando que a tecnologia está em constante mudança e o mundo é **dinâmico**, é necessário estar preparado para incorporar novas funcionalidades e integrar novos serviços de forma eficiente.

Ao mesmo tempo, devemos pensar em termos de **desempenho** e **segurança**. Tudo isso deve fazer parte da nossa decisão sobre qual arquitetura utilizar. Aliás, não precisamos adotar uma única arquitetura; podemos adotar uma combinação de arquiteturas de acordo com as diferentes partes e objetivos da nossa aplicação. Com isso, avançamos dentro do estágio de arquitetura e design.

Próximos passos

Após a análise da arquitetura, é necessário aprofundarmos na análise do design, focando nas boas práticas que garantem a construção de códigos de qualidade para a aplicação. Este é o próximo passo que iremos explorar!